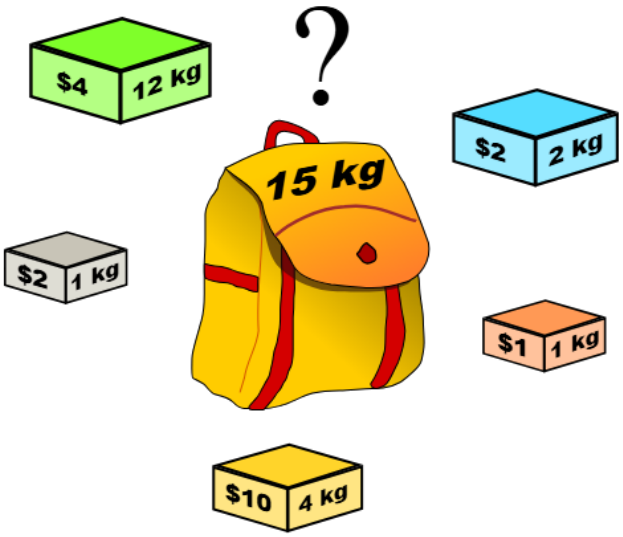
Tugas Besar 1

Penerapan Algoritma *Local Search* untuk Optimasi Pemilihan dan Penataan Paket dalam Peti Kemas

IF3170 Intelegensi Artifisial

Semester 1 Tahun 2026/2027



Disusun oleh:

Three Agents

Muhammad Aufar Rizqi Kusuma	(13524061)
Arina Azka	(13524049)
Michael James Liman	(13524106)

Laboratorium Intelegensi Buatan
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**
1.1 *Backstory* . . . 3
1.2 *Overview* Permasalahan . . . 3
2 Landasan Teori **5**
2.1 Algoritma Hill Climbing . . . 5
2.1.1 *Steepest Ascent Hill Climbing* . . . 5
2.1.2 *Hill Climbing with Sideways Move* . . . 5
2.1.3 *Stochastic Hill Climbing* . . . 5
2.1.4 *Random Restart Hill Climbing* . . . 6
2.2 Algoritma Simulated Annealing . . . 6
2.3 Genetic Algorithm . . . 8
3 Implementasi Algoritma **10**
3.1 Representasi State . . . 10
3.2 *Constraint* Permasalahan . . . 11
3.3 *Successor Function* . . . 13
3.4 *Objective Function* . . . 13
3.5 Formulasi *Initial State* . . . 14
3.6 Implementasi Hill Climbing . . . 14
3.6.1 Pembangkitan *Neighbor* dan Heuristik . . . 14
3.6.2 Parameter dan Kondisi Berhenti . . . 15
3.6.3 Metrik yang Dicatat . . . 15
3.6.4 *Source Code* . . . 15
3.6.5 Pelaksana Eksperimen dan Visualisasi . . . 18
3.7 Implementasi Simullated Annealing . . . 18
3.8 Implementasi Genetic Algorithm . . . 18
3.8.1 Konfigurasi dan Generator Acak . . . 18
3.8.2 Inisialisasi Populasi . . . 18
3.8.3 Evaluasi *Fitness* dan Penyimpanan Individu Terbaik . . . 19
3.8.4 Seleksi Induk dengan *Roulette Wheel* . . . 19
3.8.5 Crossover Satu Titik dan Perbaikan Kelayakan . . . 19
3.8.6 Mutasi Anak . . . 20
3.8.7 Pembentukan Generasi Berikutnya dan Terminasi . . . 20
4 Hasil Eksperimen dan Analisis **22**
4.1 Metodologi Eksperimen . . . 22
4.1.1 Kumpulan Instance dan Kasus Uji . . . 22
4.1.2 Protokol Perbandingan . . . 22
4.1.3 Metrik dan Analisis . . . 23
4.1.4 Studi Ablasi . . . 23
4.2 Hasil Eksperimen dan Analisis tiap Algoritma . . . 23
4.2.1 Hill Climbing . . . 23
4.3 Hasil Eksperimen dan Analisis tiap Algoritma . . . 23
4.3.1 Hill Climbing . . . 23
4.3.2 Simulated Annealing . . . 23
4.3.3 Genetic Algorithm . . . 23
4.4 Analisis Keseluruhan . . . 23

5 Kesimpulan dan Saran **24**
5.1 Kesimpulan 24
5.2 Saran 24
6 Lampiran **25**
6.1 Pembagian Tugas 25
6.2 Repositori GitHub 25

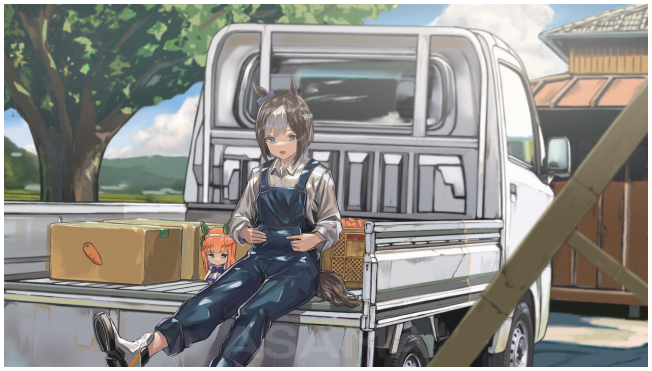
Bab 1

Pendahuluan

1.1 *Backstory*

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk uji coba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan "Lebih kilat dari Umamusume".

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.



Gambar 1.1: Special Week, driver truk handal (Sumber: Manatsu no Gin Hourui Uma C108 Doyoubi Higashi Chiku Ho-10a on X)

1.2 *Overview* Permasalahan

Tugas Besar 1 mata kuliah IF3170 Inteligensi Artifisial menugaskan mahasiswa untuk mencari konfigurasi pemuatan paket yang optimal pada sebuah truk peti kemas. Secara umum, paket memiliki ukuran, berat, dan nilai yang beragam, sedangkan ruang serta kapasitas angkut truk terbatas. Tujuan optimasi adalah memilih dan menata paket agar nilai total paket yang berhasil dimuat menjadi sebesar mungkin.

Permasalahan ini tidak hanya berkaitan dengan pemilihan paket bernilai tinggi, tetapi juga dengan kemungkinan penataannya di dalam ruang tiga dimensi. Susunan paket harus tetap layak untuk diangkut dengan memperhatikan keterbatasan ruang, kestabilan tumpukan, kondisi barang pecah-belah, dan kapasitas truk. Beragamnya kombinasi paket dan penataannya membuat pencarian konfigurasi terbaik menjadi sulit dilakukan secara menyeluruh.

Untuk mencari solusi, tugas ini meminta implementasi tiga algoritma pencarian, yaitu satu varian Hill Climbing,

Simulated Annealing, dan Genetic Algorithm. Algoritma-algoritma tersebut mengeksplorasi konfigurasi secara bertahap dari solusi awal yang dibentuk secara acak, lalu berusaha menemukan konfigurasi dengan nilai objektif yang lebih baik. Kinerja pencarian kemudian dibandingkan melalui kualitas solusi, perubahan nilai objektif selama proses, durasi, dan konsistensi hasil eksperimen.

Bab 2

Landasan Teori

2.1 Algoritma Hill Climbing

Hill climbing adalah algoritma *local search* yang bekerja pada satu *state* saja. Algoritma ini mulai dari sebuah *state* awal, membangkitkan *neighbor* dari *state* saat ini, lalu berpindah ke *neighbor* yang nilai *objective function*-nya lebih baik. Pencarian berhenti ketika tidak ada lagi *neighbor* yang lebih baik. Karena hanya menyimpan satu *state* dan tidak menyimpan jalur pencarian, kebutuhan memorinya sangat kecil dan setiap iterasinya murah. Pada permasalahan ini *objective function* dimaksimalkan, sehingga algoritma bergerak "naik" menuju nilai *value* total paket yang lebih besar.

Kelemahan utama *hill climbing* adalah tidak adanya mekanisme untuk keluar dari daerah yang buruk. Ada tiga kondisi yang menyebabkan pencarian terhenti sebelum mencapai *global optimum*.

- *Local maximum*: *state* yang lebih baik dari semua *neighbor*-nya, tetapi bukan yang terbaik secara global.
- *Plateau*: daerah datar tempat semua *neighbor* bernilai sama, sehingga tidak ada arah yang jelas untuk naik.
- *Shoulder*: *plateau* yang di ujungnya masih terdapat jalan menaik, sehingga pencarian yang berhenti terlalu cepat melewatkannya.

Keempat varian berikut dibuat untuk mengurangi kelemahan tersebut dengan cara yang berbeda.

2.1.1 *Steepest Ascent Hill Climbing*

Pada setiap iterasi, algoritma mengevaluasi sekumpulan *neighbor* dan memilih *neighbor* dengan nilai terbaik. Algoritma berpindah ke *neighbor* tersebut hanya jika nilainya *strictly* lebih besar daripada nilai *state* saat ini; jika tidak, algoritma berhenti karena telah mencapai *local optimum*. Varian ini deterministik terhadap pilihan langkahnya dan berhenti paling cepat, tetapi paling mudah terjebak pada *local maximum* dan *plateau*.

2.1.2 *Hill Climbing with Sideways Move*

Varian ini sama dengan *steepest ascent*, tetapi juga mengizinkan perpindahan ke *neighbor* yang nilainya sama dengan *state* saat ini (*sideways move*). Dengan cara ini algoritma dapat menyeberangi *plateau* dan *shoulder*. Agar tidak berputar tanpa akhir pada *plateau* yang tidak memiliki jalan keluar, jumlah *sideways move* dibatasi oleh parameter *maximum sideways move*. Pencarian dihentikan ketika jumlah *sideways move* mencapai batas tersebut.

2.1.3 *Stochastic Hill Climbing*

Algoritma memilih satu *neighbor* secara acak pada setiap iterasi dan berpindah ke *neighbor* tersebut hanya jika nilainya lebih baik. Varian ini tidak perlu mengevaluasi seluruh *neighbor* dalam satu iterasi sehingga setiap iterasinya lebih murah, dan sifat acaknya membuat jalur pencarian lebih bervariasi. Karena tidak ada kondisi

"tidak ada *neighbor* yang lebih baik" yang dapat dipastikan dari satu sampel, pencarian dihentikan setelah jumlah iterasi maksimum tercapai.

2.1.4 *Random Restart Hill Climbing*

Algoritma menjalankan *steepest ascent hill climbing* berulang kali, setiap kali dimulai dari *state* awal acak yang baru, dan menyimpan hasil terbaik dari seluruh percobaan. Peluang menemukan *global optimum* meningkat seiring bertambahnya jumlah *restart*, karena setiap *restart* mengeksplorasi daerah ruang pencarian yang berbeda. Pencarian dihentikan ketika jumlah *restart* mencapai *maximum restart*. Kelemahannya adalah biaya komputasi yang bertambah sebanding dengan jumlah *restart*.

Algorithm 1: Steepest-Ascent Hill Climbing

Data: Konfigurasi awal s_0, fungsi tetangga N, fungsi objektif f
Result: Konfigurasi optimum lokal s

```
s ← s_0
while N(s) ≠ ∅ do
    s' ← arg max_{u∈N(s)} f(u)
    if f(s') ≤ f(s) then
        return s
    end if
    s ← s'
end while
return s
```

2.2 Algoritma Simulated Annealing

Simulated Annealing adalah algoritma pencarian lokal stokastik yang meniru proses pendinginan material untuk mencari solusi bernilai objektif tinggi. Berbeda dari Hill Climbing yang hanya menerima perpindahan ke solusi lebih baik, Simulated Annealing juga dapat menerima solusi yang lebih buruk dengan peluang tertentu. Mekanisme ini membantu pencarian keluar dari optimum lokal dan mengeksplorasi bagian lain dari ruang solusi [1, 2].

Pada setiap iterasi, algoritma memilih satu solusi tetangga dan menghitung perubahan nilai objektifnya. Untuk persoalan maksimisasi, perubahan tersebut didefinisikan sebagai $\Delta E = f(s') - f(s)$, dengan s sebagai solusi saat ini dan s' sebagai kandidat tetangga. Kandidat yang meningkatkan atau mempertahankan nilai objektif selalu diterima. Kandidat yang menurunkan nilai objektif dapat diterima dengan peluang yang bergantung pada temperatur T, seperti dirumuskan pada Persamaan 2.1 [2].

$$P(\text{menerima } s') = \begin{cases} 1, & \Delta E \geq 0, \\ \exp\left(\dfrac{\Delta E}{T}\right), & \Delta E < 0. \end{cases} \tag{2.1}$$

Temperatur mengendalikan tingkat eksplorasi algoritma. Pada temperatur tinggi, penurunan nilai objektif masih mungkin diterima sehingga pencarian dapat menjelajahi ruang solusi dengan lebih luas. Ketika temperatur diturunkan, peluang menerima perpindahan yang lebih buruk ikut mengecil dan pencarian semakin berfokus pada perbaikan. Salah satu jadwal pendinginan yang umum digunakan adalah pendinginan geometris, dengan $0 < \alpha < 1$ sebagai faktor pendinginan [1].

$$T_{k+1} = \alpha T_k, \qquad 0 < \alpha < 1. \tag{2.2}$$

Keunggulan Simulated Annealing adalah kemampuannya melewati optimum lokal dengan menerima perpindahan yang sesaat menurunkan nilai objektif, tanpa harus menyimpan banyak kandidat solusi. Namun, kualitas hasil dan waktu pencarian sensitif terhadap temperatur awal, faktor pendinginan, temperatur minimum, serta mekanisme pembentukan tetangga. Jika temperatur turun terlalu cepat, pencarian masih dapat terjebak pada

optimum lokal; frekuensi kondisi ini menjadi salah satu metrik evaluasi algoritma. Karena itu, algoritma ini tidak menjamin solusi global terbaik untuk jadwal pendinginan berdurasi terbatas dan hasilnya dapat berbeda antarpercobaan [1, 2].

Simulated Annealing cocok digunakan pada ruang solusi besar dengan banyak optimum lokal, terutama ketika evaluasi solusi cukup murah dan solusi mendekati optimal sudah bermanfaat. Dalam persoalan pemuatan paket, pencarian dimulai dari konfigurasi acak. Pada setiap iterasi, satu tetangga dibentuk melalui operasi pertukaran paket, pemindahan paket, atau rotasi. Algoritma menerima kandidat yang meningkatkan nilai objektif, dan sesekali menerima kandidat yang menurunkannya sesuai peluang pada Persamaan 2.1. Temperatur kemudian dikurangi berdasarkan jadwal pendinginan. Nilai $\exp(\Delta E/T)$ menjadi ukuran peluang penerimaan kandidat yang lebih buruk, sesuai metrik yang diminta spesifikasi untuk dianalisis terhadap iterasi.

Pseudocode berikut menggunakan pendinginan geometris dan berhenti ketika temperatur mencapai nilai minimum. Variabel s_{best} menyimpan konfigurasi bernilai tertinggi yang ditemukan, meskipun proses menerima perpindahan sementara ke konfigurasi yang lebih buruk.

Algorithm 2: Simulated Annealing

Data: Solusi awal s_0, fungsi tetangga N, fungsi objektif f, parameter $T_0 > T_{\min} > 0$ dan $0 < \alpha < 1$
Result: Solusi terbaik yang ditemukan s_{best}

```
s ← s_0, s_best ← s_0, T ← T_0
while T > T_min do
    if N(s) = ∅ then
        return s_best
    end if
    Pilih s' secara acak dari N(s)
    ΔE ← f(s') − f(s)
    if ΔE ≥ 0 then
        s ← s'
    else
        Bangkitkan u secara acak dari [0, 1]
        if u < exp(ΔE/T) then
            s ← s'
        end if
    end if
    if f(s) > f(s_best) then
        s_best ← s
    end if
    T ← αT
end while
return s_best
```

Algorithm 3: Simulated Annealing

Data: Solusi awal s_0, fungsi tetangga N, fungsi objektif f, parameter $T_0 > T_{\min} > 0$ dan $0 < \alpha < 1$
Result: Solusi terbaik yang ditemukan s_{best}

```
s ← s_0, s_best ← s_0, T ← T_0
while T > T_min do
    if N(s) = ∅ then
        return s_best
    end if
    Pilih s' secara acak dari N(s)
    ΔE ← f(s') − f(s)
    if ΔE ≥ 0 then
        s ← s'
    else
```

Bangkitkan u secara acak dari $[0, 1]$
if $u < \exp(\Delta E/T)$ **then**
$s \leftarrow s'$
end if
end if
if $f(s) > f(s_{\text{best}})$ **then**
$s_{\text{best}} \leftarrow s$
end if
$T \leftarrow \alpha T$
end while
return s_{best}

2.3 Genetic Algorithm

Genetic Algorithm adalah metode pencarian berbasis populasi yang memperbarui kumpulan kandidat solusi melalui mekanisme yang terinspirasi oleh seleksi alam. Setiap kandidat disebut individu, sedangkan nilai objektifnya berperan sebagai ukuran fitness. Pada tiap generasi, algoritma memilih individu sebagai induk, menggabungkan bagian solusi melalui crossover, lalu menerapkan mutasi secara acak untuk membentuk populasi baru [1, 3].

Karakteristik Genetic Algorithm adalah pencarian atas banyak kandidat secara bersamaan dan penggunaan operator stokastik untuk menghasilkan generasi berikutnya. Seleksi roulette memberi peluang lebih besar kepada individu dengan fitness lebih tinggi. Jika fitness individu ke-i adalah f_i dan bobot seleksinya w_i, peluang memilih individu tersebut dirumuskan sebagai

$$P(i) = \frac{w_i}{\sum_{j=1}^{n} w_j}. \tag{2.3}$$

Untuk persoalan pemuatan paket dengan nilai objektif nonnegatif, bobot seleksi dapat menggunakan nilai objektif. Jika seluruh bobot bernilai nol, seleksi dilakukan secara seragam. Crossover satu titik menggabungkan gen dari dua induk, sementara mutasi mengubah satu gen dengan peluang tertentu. Kedua operator harus menghasilkan individu yang memenuhi batasan pemuatan paket [3].

Keunggulan Genetic Algorithm adalah kemampuannya mengeksplorasi beberapa wilayah ruang solusi secara paralel dan mengurangi ketergantungan pencarian pada satu konfigurasi awal. Keragaman populasi dapat membantu pencarian menghindari optimum lokal. Namun, hasil dan waktu pencarian bergantung pada ukuran populasi, jumlah generasi, operator, serta parameter seperti laju mutasi. Algoritma ini tidak menjamin solusi global terbaik dalam jumlah generasi terbatas, dan setiap generasi memerlukan evaluasi banyak kandidat [1, 3].

Genetic Algorithm sesuai untuk ruang solusi besar dengan representasi kandidat yang dapat digabungkan dan dimutasi, khususnya ketika solusi layak dapat dipulihkan setelah operator diterapkan. Pada persoalan pemuatan paket, satu individu merepresentasikan penempatan seluruh paket. Populasi awal memuat konfigurasi awal yang valid dan konfigurasi valid lain yang dibentuk secara acak. Di setiap generasi, individu dievaluasi dengan fungsi objektif, induk dipilih menggunakan roulette, kemudian crossover satu titik dan mutasi dilakukan untuk membentuk populasi berikutnya. Individu terbaik disimpan terpisah sepanjang pencarian agar solusi terbaik tetap tersedia meskipun tidak diwariskan ke generasi baru.

Pseudocode berikut menggunakan seleksi roulette, crossover satu titik dengan perbaikan kelayakan, dan mutasi satu gen dengan peluang μ. Pencarian berhenti setelah jumlah generasi yang ditentukan dan mengembalikan individu terbaik yang pernah dievaluasi.

Algorithm 4: Genetic Algorithm

Data: Solusi awal valid s_0, fungsi objektif f, ukuran populasi n, jumlah generasi G, laju mutasi μ
Result: Individu terbaik yang ditemukan s_{best}
$P \leftarrow$ populasi valid berukuran n yang memuat s_0
$s_{\text{best}} \leftarrow s_0$, $f_{\text{best}} \leftarrow -\infty$
for $g \leftarrow 0$ sampai G **do**
Evaluasi $f(x)$ untuk setiap $x \in P$
$x_{\text{best}} \leftarrow \arg\max_{x \in P} f(x)$
if $f(x_{\text{best}}) > f_{\text{best}}$ **then**
$s_{\text{best}} \leftarrow x_{\text{best}}$, $f_{\text{best}} \leftarrow f(x_{\text{best}})$

```
    end if
    if g < G then
      P_baru ← ∅
      while |P_baru| < n do
        Pilih induk p1 dan p2 dari P dengan seleksi roulette
        c ← crossover satu titik (p1, p2) dan perbaiki agar valid
        Dengan peluang μ, mutasikan satu gen pada c dan pertahankan validitasnya
        Tambahkan c ke P_baru
      end while
      P ← P_baru
    end if
  end for
  return s_best
```

Algorithm 5: Genetic Algorithm

Data: Solusi awal valid s_0, fungsi objektif f, ukuran populasi n, jumlah generasi G, laju mutasi μ
Result: Individu terbaik yang ditemukan s_{best}

```
P ← populasi valid berukuran n yang memuat s_0
s_best ← s_0, f_best ← −∞
for g ← 0 sampai G do
  Evaluasi f(x) untuk setiap x ∈ P
  x_best ← arg max_{x∈P} f(x)
  if f(x_best) > f_best then
    s_best ← x_best, f_best ← f(x_best)
  end if
  if g < G then
    P_baru ← ∅
    while |P_baru| < n do
      Pilih induk p1 dan p2 dari P dengan seleksi roulette
      c ← crossover satu titik (p1, p2) dan perbaiki agar valid
      Dengan peluang μ, mutasikan satu gen pada c dan pertahankan validitasnya
      Tambahkan c ke P_baru
    end while
    P ← P_baru
  end if
end for
return s_best
```

Bab 3

Implementasi Algoritma

3.1 Representasi State

State merepresentasikan satu konfigurasi pemuatan paket pada sebuah truk. Implementasi menyimpan data paket dan truk secara terpisah, lalu menghubungkan setiap paket dengan posisinya melalui pemetaan dari ID paket ke objek penempatan. Nilai penempatan `None` menandakan paket berada di luar truk, sehingga state tetap mencatat seluruh paket, baik yang dimuat maupun tidak.
Objek `Package` menyimpan ID, dimensi awal, nilai, berat, status mudah pecah, dan ETA. Dimensi disimpan sebagai tiga bilangan bulat, sedangkan orientasi adalah permutasi indeks dimensi yang menentukan urutan ukuran setelah rotasi. Ada enam kemungkinan orientasi, sesuai seluruh permutasi indeks tiga dimensi. Objek `Truck` menyimpan dimensi ruang muat serta kapasitas berat maksimum. Objek `Placement` menggabungkan koordinat sudut paket, dimensi setelah orientasi diterapkan, dan orientasinya. Kode 3.1 memperlihatkan struktur data inti dari `src/model.py`; pemeriksaan validasi dihilangkan dari cuplikan agar fokus pada field representasinya.

Kode 3.1: Struktur inti model state

```
from dataclasses import dataclass
from itertools import permutations
from typing import Mapping, TypeAlias
Dimensions: TypeAlias = tuple[int, int, int]
Coordinate: TypeAlias = tuple[int, int, int]
Orientation: TypeAlias = tuple[int, int, int]
ORIENTATIONS = tuple(permutations((0, 1, 2)))
@dataclass(frozen=True, slots=True)
class Package:
    id: str
    dimensions: Dimensions
    value: int
    weight: int
    fragile: bool
    eta: int
@dataclass(frozen=True, slots=True)
class Truck:
    dimensions: Dimensions
    max_capacity: int
@dataclass(frozen=True, slots=True)
class Placement:
    coordinate: Coordinate
    oriented_dimensions: Dimensions
    orientation: Orientation = (0, 1, 2)
@dataclass(frozen=True, slots=True)
class State:
    packages: tuple[Package, ...]
    truck: Truck
    placements: Mapping[str, Placement | None]
```

Secara matematis, satu state dituliskan sebagai $S = (P, T, \phi)$, dengan $P = (p_1, \ldots, p_n)$ sebagai kumpulan paket, $T = (\mathbf{L}, C)$ sebagai truk berdimensi $\mathbf{L}$ dan berkapasitas maksimum C, serta ϕ sebagai pemetaan penempatan. Jika paket p_i memiliki ID id_i dan dimensi $\mathbf{d}_i$, maka representasinya adalah

$$
\begin{aligned}
& I = \{\mathrm{id}_1, \ldots, \mathrm{id}_n\}, \\
& \phi : I \longrightarrow \{\perp\} \cup \left\{ (\mathbf{x}_i, \mathbf{d}_i^{\pi_i}, \pi_i) \;\middle|\; \mathbf{x}_i \in \mathbb{Z}^3_{\geq 0},\ \pi_i \in S_3 \right\}, \\
& \mathbf{d}_i^{\pi_i} = (d_{i,\pi_i(1)}, d_{i,\pi_i(2)}, d_{i,\pi_i(3)}).
\end{aligned} \tag{3.1}
$$

Di sini $\perp$ menyatakan paket tidak dimuat, $\mathbf{x}_i$ adalah koordinat sudut paket di dalam truk, dan S_3 adalah himpunan enam permutasi sumbu yang merepresentasikan orientasi. Domain I memastikan setiap paket memiliki tepat satu entri penempatan. Persamaan 3.1 menjelaskan bentuk representasi; batas ruang, kapasitas, dan batasan fisik lainnya menentukan apakah state tersebut layak.

Pemetaan `placements` harus memiliki tepat satu entri untuk setiap ID paket. Konstruktor `State` memeriksa keunikan ID, kelengkapan kunci pemetaan, dan kecocokan dimensi hasil orientasi dengan dimensi paket. Fungsi objektif state menjumlahkan nilai paket yang memiliki penempatan, sehingga paket di luar truk tidak berkontribusi terhadap nilai solusi.

Contoh berikut menunjukkan sebuah paket di dalam truk dan satu paket yang tidak dimuat. Koordinat menggunakan urutan sumbu x, y, dan z, sedangkan koordinat $(0, 0, 0)$ menandakan sudut bawah paket berada di titik asal ruang muat.

Kode 3.2: Contoh pembentukan state pemuatan

```
from src.model import Package, Placement, State, Truck

package = Package("P001", (3, 4, 1), 60, 5, False, 3)
outside_package = Package("P002", (1, 1, 1), 10, 2, False, 1)
truck = Truck((10, 20, 5), 500)

state = State(
    packages=(package, outside_package),
    truck=truck,
    placements={
        "P001": Placement((0, 0, 0), (3, 4, 1)),
        "P002": None,
    },
)
```

3.2 *Constraint* Permasalahan

State layak apabila seluruh syarat berikut terpenuhi. Misalkan $I^+ = \{i \in I \mid \phi(i) \neq \perp\}$ adalah himpunan ID paket yang dimuat. Untuk setiap $i \in I^+$, penempatannya ditulis sebagai $\phi(i) = (\mathbf{x}_i, \mathbf{d}_i, \pi_i)$, dengan $\mathbf{x}_i = (x_i, y_i, z_i)$ sebagai koordinat dan $\mathbf{d}_i = (w_i, d_i, h_i)$ sebagai dimensi setelah rotasi. Syarat kelayakan dituliskan dalam enumerasi berikut.

(1) Setiap paket harus seluruhnya berada di dalam ruang muat truk:

$$\mathbf{0} \leq \mathbf{x}_i, \qquad \mathbf{x}_i + \mathbf{d}_i \leq \mathbf{L}, \qquad \forall i \in I^+.$$

(2) Setiap pasangan paket harus terpisah pada setidaknya satu sumbu, sehingga keduanya tidak saling bertumpang tindih. Di sini $d_{i,k}$ adalah ukuran paket i pada sumbu k:

$$\exists k \in \{x, y, z\} : \quad x_{i,k} + d_{i,k} \leq x_{j,k} \ \vee \ x_{j,k} + d_{j,k} \leq x_{i,k}, \qquad \forall i, j \in I^+,\ i \neq j.$$

(3) Jumlah berat semua paket yang dimuat tidak boleh melebihi kapasitas truk C. Simbol w_i^{paket} menyatakan berat paket, bukan ukuran pada sumbu x:

$$\sum_{i \in I^+} w_i^{\text{paket}} \leq C.$$

(4) Paket yang berada di atas lantai truk harus ditopang oleh setidaknya satu paket lain yang tidak rapuh. Bagian atas paket penopang harus sejajar dengan dasar paket yang ditopang, dan proyeksi alas keduanya harus beririsan:

$$z_i > 0 \Rightarrow \exists j \in I^+,\ j \neq i : \quad z_j + h_j = z_i, \quad \neg \text{rapuh}(j), \quad \text{overlap}_{xy}(i, j), \qquad \forall i \in I^+.$$

Predikat overlap_{xy} bernilai benar ketika proyeksi alas dua paket beririsan dengan luas positif. Karena itu, implementasi tidak menganggap kontak yang hanya terjadi pada tepi atau satu titik sebagai dukungan, meskipun

spesifikasi menyebut dukungan pada setidaknya satu titik alas. Paket di lantai truk ($z_i = 0$) tidak memerlukan paket penopang. Kode 3.3 menunjukkan pemeriksaan yang dipakai oleh `validate_state`; fungsi bantu irisan satu dimensi dan alas juga disertakan.

Kode 3.3: Validasi batas ruang, tabrakan, kapasitas, dan dukungan

```
from itertools import combinations

from src.model import Package, Placement, State

def _overlap_1d(start_a, size_a, start_b, size_b):
    return start_a < start_b + size_b and start_b < start_a + size_a

def _overlap_xy(a, b):
    return (
        _overlap_1d(a.coordinate[0], a.oriented_dimensions[0],
                    b.coordinate[0], b.oriented_dimensions[0])
        and _overlap_1d(a.coordinate[1], a.oriented_dimensions[1],
                        b.coordinate[1], b.oriented_dimensions[1])
    )

def _overlap_3d(a, b):
    return _overlap_xy(a, b) and _overlap_1d(
        a.coordinate[2], a.oriented_dimensions[2],
        b.coordinate[2], b.oriented_dimensions[2]
    )

def validate_state(state):
    errors = []
    placed = [
        (package, placement)
        for package in state.packages
        if (placement := state.placements[package.id]) is not None
    ]

    for package, placement in placed:
        if any(
            position + size > limit
            for position, size, limit in zip(
                placement.coordinate, placement.oriented_dimensions,
                state.truck.dimensions
            )
        ):
            errors.append(f"package {package.id!r} is outside the truck")

    for (package_a, placement_a), (package_b, placement_b) in combinations(placed, 2):
        if _overlap_3d(placement_a, placement_b):
            errors.append(f"packages {package_a.id!r} and {package_b.id!r} overlap")

    total_weight = sum(package.weight for package, _ in placed)
    if total_weight > state.truck.max_capacity:
        errors.append("truck capacity is exceeded")

    for package, placement in placed:
        if placement.coordinate[2] == 0:
            continue
        supported = any(
            not support_package.fragile
            and support_placement.coordinate[2]
                + support_placement.oriented_dimensions[2]
                == placement.coordinate[2]
            and _overlap_xy(placement, support_placement)
            for support_package, support_placement in placed
            if support_package.id != package.id
        )
        if not supported:
            errors.append(f"package {package.id!r} is unsupported")

    return errors
```

3.3 *Successor Function*

Fungsi suksesor membangkitkan kandidat dengan menerapkan salah satu operator berikut secara acak:

(1) *Move* memilih satu paket. Jika paket sudah dimuat, orientasinya dipertahankan; jika masih di luar truk, orientasi dipilih secara acak. Paket kemudian ditempatkan pada koordinat baru yang masih berada dalam rentang truk.

(2) *Rotate* memilih satu paket yang sedang dimuat, lalu menukar dua sumbu orientasinya (rotasi 90 derajat) tanpa mengubah koordinat paket.

(3) *Swap* memilih dua paket dan menukar status pemuatan atau posisi keduanya. Dimensi penempatan disesuaikan dengan ukuran dan orientasi masing-masing paket.

Operator *swap* hanya tersedia jika state memiliki setidaknya dua paket. Misalkan $\mathcal{O}(S)$ adalah himpunan operator yang tersedia pada state S. Himpunan suksesor layak didefinisikan sebagai

$$N(S) = \{S' \mid S' = o(S),\ o \in \mathcal{O}(S),\ S' \neq S,\ \text{validate}(S') = \emptyset\}. \tag{3.2}$$

Pada operator *move*, ketinggian dipilih dari lantai truk atau permukaan atas paket lain. Setiap kandidat harus berbeda dari state asal dan lolos validasi. Implementasi mencoba paling banyak 100 kandidat, lalu mengembalikan kandidat layak pertama; jika tidak ada paket atau tidak ditemukan kandidat yang layak, fungsi mengembalikan state asal. Cuplikan 3.4 memperlihatkan pemilihan operator dan penyaringan tersebut.

Kode 3.4: Pemilihan kandidat suksesor yang valid

```
from random import Random

def random_neighbor(state, rng=None, max_attempts=100):
    if max_attempts < 1 or not state.packages:
        return state
    rng = rng or Random()
    operations = [_move_candidate, _rotate_candidate]
    if len(state.packages) > 1:
        operations.append(_swap_candidate)

    for _ in range(max_attempts):
        candidate = rng.choice(operations)(state, rng)
        if candidate is not state and _valid(candidate):
            return candidate
    return state
```

3.4 *Objective Function*

Fungsi objektif memberi nilai pada state berdasarkan total nilai paket yang berhasil dimuat. Untuk state $S = (P, T, \phi)$, definisikan $I^+(S) = \{i \in P \mid \phi(i) \neq \perp\}$ sebagai paket yang berada di dalam truk. Nilai state dan tujuan optimasinya adalah

$$f(S) = \sum_{i \in I^+(S)} v_i, \tag{3.3}$$

$$S^* \in \arg\max_{S \in \mathcal{F}} f(S), \tag{3.4}$$

dengan v_i adalah nilai paket i dan $\mathcal{F}$ adalah himpunan state yang memenuhi seluruh constraint pada bagian sebelumnya. Dengan demikian, paket yang berada di luar truk tidak menyumbang nilai, sementara paket dengan nilai lebih tinggi meningkatkan nilai objektif jika dapat dimuat tanpa melanggar constraint. Fungsi ini tidak memberi bonus langsung untuk ruang kosong atau penalti berat; berat hanya membatasi state melalui constraint kapasitas. Implementasi menghitung jumlah nilai paket yang memiliki penempatan, seperti ditunjukkan pada Kode 3.5.

Kode 3.5: Perhitungan nilai objektif sebuah state

```
def objective(self):
    return sum(
        package.value
        for package in self.packages
        if self.placements[package.id] is not None
    )
```

3.5 Formulasi *Initial State*

Program membentuk state awal secara konstruktif dan acak, bukan dengan mengambil sampel seragam dari seluruh state layak. Mula-mula semua paket dianggap berada di luar truk, kemudian paket diproses dalam urutan acak. Proses untuk paket ke-k dapat diringkas sebagai

(1) Atur penempatan awal setiap paket menjadi $\perp$, yang menandakan paket berada di luar truk.
(2) Acak urutan paket yang akan dicoba.
(3) Untuk setiap paket, coba hingga 50 kali secara default. Pada tiap percobaan, pilih orientasi secara acak, lalu pilih koordinat yang muat di dalam truk.
(4) Terima penempatan pertama yang menghasilkan state valid. Jika semua percobaan gagal, biarkan paket tersebut di luar truk dan lanjutkan ke paket berikutnya.

Secara formal, jika σ adalah urutan acak paket, penempatan dimulai dengan $\phi_0(i) = \perp$ untuk setiap $i \in P$. Pada langkah k, kandidat ϕ'_k mengganti penempatan paket σ_k; kandidat diterima hanya jika validate$(P, T, \phi'_k) = \emptyset$. Setelah seluruh paket diproses, pemetaan yang terbentuk menjadi state awal $S_0 = (P, T, \phi)$. Koordinat horizontal dipilih acak dari rentang integer yang muat, sedangkan koordinat z dipilih dari lantai truk atau ketinggian permukaan atas paket yang sudah ditempatkan. Kode 3.6 memperlihatkan inti proses tersebut.

Kode 3.6: Pembentukan state awal acak yang valid

```python
def random_initial_state(packages, truck, rng=None, attempts_per_package=50):
    rng = rng or Random()
    placements = {package.id: None for package in packages}
    order = list(packages)
    rng.shuffle(order)

    for package in order:
        for _ in range(attempts_per_package):
            orientation = rng.choice(ORIENTATIONS)
            dimensions = package.oriented_dimensions(orientation)
            if any(size > limit for size, limit in zip(dimensions, truck.dimensions)):
                continue
            candidate_placements = placements.copy()
            candidate_placements[package.id] = _candidate_position(
                package, orientation, truck, placements, rng
            )
            candidate = _make_state(packages, truck, candidate_placements)
            if _valid(candidate):
                placements = candidate_placements
                break

    return _make_state(packages, truck, placements)
```

Karena paket diproses satu per satu dengan jumlah percobaan terbatas, hasil state awal bergantung pada urutan dan bilangan acak yang digunakan; paket yang gagal ditempatkan tidak menghalangi paket berikutnya untuk dicoba. Strategi ini selalu mempertahankan state valid, tetapi tidak menjamin nilai awal maksimum atau bahwa setiap paket yang sebenarnya dapat dimuat akan berhasil ditempatkan.

3.6 Implementasi Hill Climbing

Seluruh varian *hill climbing* diimplementasikan pada berkas `src/algorithms/hill_climbing/algorithm.py` di atas representasi *state*, validator *constraint*, dan fungsi *neighbor* yang sama dengan algoritma lainnya (`State`, `validate_state`, dan `random_neighbor`). Dengan demikian seluruh algoritma menerima masukan dan mengembalikan keluaran yang seragam, yaitu objek `SearchResult` yang memuat *state* awal, *state* akhir, nilai *objective function* akhir, riwayat nilai per iterasi, durasi pencarian, dan metrik khusus algoritma.

3.6.1 Pembangkitan *Neighbor* dan Heuristik

Ruang *neighbor* pada permasalahan ini sangat besar karena koordinat paket berupa bilangan bulat, sehingga seluruh *neighbor* tidak dapat dienumerasi pada setiap iterasi. Sebagai gantinya, setiap *neighbor* dibangkitkan oleh `random_neighbor`, yang secara acak memilih salah satu dari tiga operasi (menukar dua paket, memindahkan paket ke koordinat lain, atau merotasi paket 90°) dan hanya mengembalikan *state* yang lolos validasi *constraint*. Heuristik yang kami gunakan adalah *steepest* berbasis sampel: pada setiap iterasi dibangkitkan `neighbors_per_iteration` (bawaan 30) *neighbor* valid, kemudian *neighbor* dengan nilai terbaik dianggap sebagai *neighbor* terbaik. Konsekuensinya, varian *steepest* dan *sideways* tidak menjamin menemukan *neighbor* terbaik yang sesungguhnya,

tetapi peluangnya naik seiring bertambahnya ukuran sampel. Varian *stochastic* memakai tepat satu *neighbor* per iterasi.

3.6.2 Parameter dan Kondisi Berhenti

Parameter	Bawaan	Keterangan
`variant`	`steepest`	`steepest`, `sideways`, `stochastic`, atau `random_restart`
`neighbors_per_iteration`	30	Jumlah *neighbor* yang disampel per iterasi (tidak dipakai *stochastic*)
`max_iterations`	1000	Batas iterasi; menjadi kondisi berhenti pada *stochastic* dan batas pengaman pada varian lain
`max_sideways`	100	Batas *sideways move* (varian *sideways*)
`max_restarts`	10	Jumlah *restart* setelah percobaan pertama (varian *random restart*)
`seed`	—	Seed pembangkit acak agar eksperimen dapat diulang

Tabel 3.1: Parameter algoritma *hill climbing*

- *Steepest*: berhenti ketika *neighbor* terbaik dari sampel tidak lebih baik daripada *state* saat ini.
- *Sideways*: sama seperti *steepest*, tetapi *neighbor* bernilai sama diterima selama jumlah *sideways move* belum mencapai `max_sideways`.
- *Stochastic*: berhenti setelah `max_iterations` iterasi. Iterasi yang *neighbor*-nya ditolak tetap dihitung.
- *Random restart*: menjalankan *steepest* dari *state* awal yang diberikan, lalu dari `max_restarts` *state* acak baru, sehingga total terdapat `max_restarts` $+1$ pendakian. Hasil terbaik dari seluruh pendakian dikembalikan.

3.6.3 Metrik yang Dicatat

Setiap *run* mengembalikan `HillClimbingMetrics` yang berisi varian yang dipakai, banyak iterasi hingga berhenti, jumlah dan batas *sideways move*, jumlah dan batas *restart*, serta banyak iterasi pada setiap *restart*. Riwayat nilai *objective function* dicatat pada setiap iterasi (elemen pertama adalah nilai *state* awal) dan digunakan untuk membuat plot nilai *objective function* terhadap banyak iterasi. Durasi pencarian diukur dengan `perf_counter`.

3.6.4 *Source Code*

Fungsi `_climb` menjalankan satu pendakian untuk varian *steepest*, *sideways*, atau *stochastic*. Fungsi `hill_climbing` memvalidasi parameter, memilih varian, menangani pengulangan pada *random restart*, lalu menyusun `SearchResult`.

Kode 3.7: Fungsi bantu dan `_climb` pada `algorithm.py`

```
from math import isfinite
from random import Random
from time import perf_counter

from src.contracts import ObjectiveFunction, resolve_objective
from src.model import State
from src.neighbors import random_initial_state, random_neighbor
from src.results import HillClimbingMetrics, ObjectiveValue, SearchResult

VARIANTS = ("steepest", "sideways", "stochastic", "random_restart")


def _check_int(value, name: str, minimum: int) -> None:
    if type(value) is not int or value < minimum:
        raise ValueError(f"{name} must be an integer >= {minimum}")


def _score(objective: ObjectiveFunction[State], state: State) -> ObjectiveValue:
    score = objective(state)
    if isinstance(score, bool) or not isinstance(score, (int, float)) or not isfinite(score):
```

```
        raise ValueError("objective_fn must return finite integer or float values")
    return score


def _climb(
    start: State,
    objective: ObjectiveFunction[State],
    rng: Random,
    *,
    variant: str,
    neighbors_per_iteration: int,
    max_iterations: int,
    max_sideways: int,
) -> tuple[State, ObjectiveValue, list[ObjectiveValue], int, int]:
    """One climb. Returns (state, score, history, iterations, sideways_moves).

    history[0] is the start score; one entry is appended per iteration.
    """
    current, current_score = start, _score(objective, start)
    history = [current_score]
    iterations = sideways = 0

    while iterations < max_iterations:
        if variant == "stochastic":
            candidates = [random_neighbor(current, rng)]
        else:
            candidates = [random_neighbor(current, rng) for _ in range(neighbors_per_iteration)]
        scored = [(_score(objective, c), c) for c in candidates]
        best_score, best = max(scored, key=lambda pair: pair[0])

        if best_score > current_score:
            current, current_score = best, best_score
        elif variant == "sideways" and best_score == current_score and sideways < max_sideways:
            current = best
            sideways += 1
        elif variant == "stochastic":
            iterations += 1  # a rejected random neighbor still costs an iteration
            history.append(current_score)
            continue
        else:
            break  # local optimum (or sideways budget exhausted)
        iterations += 1
        history.append(current_score)

    return current, current_score, history, iterations, sideways
```

Kode 3.8: Fungsi `hill_climbing` pada `algorithm.py`

```
def hill_climbing(
    initial_state: State,
    *,
    seed: int,
    objective_fn: ObjectiveFunction[State] | None = None,
    variant: str = "steepest",
    neighbors_per_iteration: int = 30,
    max_iterations: int = 1000,
    max_sideways: int = 100,
    max_restarts: int = 10,
    objective_name: str | None = None,
    run_index: int = 1,
) -> SearchResult[State]:
    """Run one hill-climbing variant and return a shared-contract SearchResult."""
    if variant not in VARIANTS:
        raise ValueError(f"variant must be one of {VARIANTS}")
    if type(seed) is not int:
        raise ValueError("seed must be an integer")
    _check_int(neighbors_per_iteration, "neighbors_per_iteration", 1)
    _check_int(max_iterations, "max_iterations", 0)
    _check_int(max_sideways, "max_sideways", 0)
    _check_int(max_restarts, "max_restarts", 0)
    _check_int(run_index, "run_index", 1)
```

```
    started_at = perf_counter()
    rng = Random(seed)
    objective = resolve_objective(initial_state, objective_fn)
    climb_kwargs = dict(
        variant="steepest" if variant == "random_restart" else variant,
        neighbors_per_iteration=neighbors_per_iteration,
        max_iterations=max_iterations,
        max_sideways=max_sideways,
    )

    if variant == "random_restart":
        # Restart 0 uses the supplied initial state; each later one a fresh random state.
        best_state, best_score = initial_state, None
        history: list[ObjectiveValue] = []
        per_restart: list[int] = []
        for restart in range(max_restarts + 1):
            start = (
                initial_state
                if restart == 0
                else random_initial_state(initial_state.packages, initial_state.truck, rng)
            )
            state, score, hist, iters, _ = _climb(start, objective, rng, **climb_kwargs)
            history.extend(hist)
            per_restart.append(iters)
            if best_score is None or score > best_score:
                best_state, best_score = state, score
        final_state, final_score = best_state, best_score
        iterations, sideways = sum(per_restart), 0
        restarts = max_restarts
    else:
        final_state, final_score, history, iterations, sideways = _climb(
            initial_state, objective, rng, **climb_kwargs
        )
        per_restart, restarts = [], 0

    parameters: dict[str, int | float | str | bool] = {
        "variant": variant,
        "max_iterations": max_iterations,
    }
    if variant != "stochastic":
        parameters["neighbors_per_iteration"] = neighbors_per_iteration
    if variant == "sideways":
        parameters["max_sideways"] = max_sideways
    if variant == "random_restart":
        parameters["max_restarts"] = max_restarts

    return SearchResult(
        algorithm=f"hill_climbing_{variant}",
        objective_name=objective_name or ("total_value" if objective_fn is None else "custom"),
        initial_state=initial_state,
        final_state=final_state,
        objective_value=final_score,
        objective_history=tuple(history),
        duration_seconds=perf_counter() - started_at,
        metrics=HillClimbingMetrics(
            variant=variant,
            iterations=iterations,
            sideways_moves=sideways,
            maximum_sideways_moves=max_sideways if variant == "sideways" else None,
            restarts=restarts,
            maximum_restarts=max_restarts if variant == "random_restart" else None,
            iterations_per_restart=tuple(per_restart),
        ),
        seed=seed,
        run_index=run_index,
        parameters=parameters,
    )
```

Kode 3.9: Fungsi pembungkus tiap varian pada `algorithm.py`

```
def steepest_ascent_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="steepest", **kw)
```

```
def sideways_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="sideways", **kw)


def stochastic_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="stochastic", **kw)


def random_restart_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="random_restart", **kw)
```

3.6.5 Pelaksana Eksperimen dan Visualisasi

Berkas `src/experiments/hill_climbing.py` menjalankan setiap varian sebanyak tiga kali dengan tiga *seed* berbeda. *State* awal setiap *run* dibangkitkan secara acak dari *seed* tersebut. Untuk setiap *run*, program menyimpan berkas `result.json`, plot nilai *objective function* terhadap iterasi, dan visualisasi tiga dimensi *state* awal dan akhir. Ringkasan lintas *run* (rata-rata dan simpangan baku nilai akhir, durasi, dan iterasi) disimpan pada `summary.json` dan `summary.png`. Program dijalankan melalui `src/experiments/run_hill_climbing.py`.

3.7 Implementasi Simullated Annealing

3.8 Implementasi Genetic Algorithm

Implementasi Genetic Algorithm pada proyek ini menyimpan tiap individu sebagai objek `State`. Karena itu, satu gen berkorespondensi dengan penempatan satu paket, termasuk status paket di luar truk. Alur algoritma dipisah ke beberapa modul agar pembentukan populasi, seleksi, crossover, mutasi, dan loop generasi dapat dijelaskan serta diuji secara terpisah. Uraian berikut mengikuti urutan eksekusi `genetic_algorithm`.

3.8.1 Konfigurasi dan Generator Acak

Pemanggil menyediakan state awal, seed, ukuran populasi, dan jumlah generasi. Laju mutasi secara default adalah 0,05, seleksi default menggunakan *roulette wheel*, dan crossover diaktifkan. Seed dipakai untuk menginisialisasi satu generator acak yang digunakan bersama oleh seluruh tahap, sehingga run dengan masukan dan seed yang sama dapat direproduksi. Fungsi objektif dapat menggunakan metode `State.objective` atau fungsi objektif lain yang diberikan pemanggil.

Kode 3.10: Menyiapkan pencarian GA

```
rng = Random(seed)
objective = resolve_objective(initial_state, objective_fn)
population = initialize_population(initial_state, population_size, rng)
```

3.8.2 Inisialisasi Populasi

Populasi pertama harus berisi state valid dan selalu menyertakan `initial_state` sebagai individu pertama. Anggota lainnya dibentuk menggunakan generator state awal acak yang dijelaskan sebelumnya. Dengan cara ini, algoritma memulai pencarian dari solusi yang diberikan sekaligus mempertimbangkan konfigurasi alternatif. Fungsi `initialize_population` memastikan ukuran populasi positif dan menolak state awal yang melanggar constraint.

Kode 3.11: Membangun populasi awal

```
def initialize_population(initial_state, population_size, rng):
    if type(population_size) is not int or population_size < 1:
        raise ValueError("population_size must be a positive integer")
    violations = validate_state(initial_state)
    if violations:
        raise ValueError(f"initial_state is invalid: {'; '.join(violations)}")

    return (
        initial_state,
        *(
            random_initial_state(initial_state.packages, initial_state.truck, rng)
            for _ in range(population_size - 1)
```

```
        ),
    )
```

3.8.3 Evaluasi *Fitness* dan Penyimpanan Individu Terbaik

Pada awal setiap generasi, nilai objektif setiap individu dihitung sebagai *fitness*. Implementasi juga memastikan setiap nilai berupa bilangan hingga. Individu dengan nilai tertinggi pada generasi tersebut dicatat, begitu pula rata-rata fitness populasi. Di luar populasi yang sedang berjalan, `best_state` dan `best_score` menyimpan solusi terbaik yang pernah ditemukan, sehingga solusi tersebut tetap dapat dikembalikan walaupun tidak terpilih menjadi induk atau tidak muncul pada generasi berikutnya.

Kode 3.12: Evaluasi populasi dan pembaruan solusi terbaik

```
fitnesses = _evaluate_population(population, objective)
generation_best_index = max(
    range(len(population)), key=fitnesses.__getitem__
)
generation_best_score = fitnesses[generation_best_index]
best_history.append(generation_best_score)
average_history.append(sum(fitnesses) / len(fitnesses))

if best_score is None or generation_best_score > best_score:
    best_state = population[generation_best_index]
    best_score = generation_best_score
```

3.8.4 Seleksi Induk dengan *Roulette Wheel*

Untuk menghasilkan anak, program memilih dua induk dari populasi saat ini. Pada konfigurasi standar, seleksi menggunakan *roulette wheel*: individu dengan fitness lebih tinggi mendapat peluang lebih besar untuk dipilih, tetapi individu lain tetap berpeluang terpilih. Karena fungsi objektif pemuatan paket bernilai nonnegatif, nilainya langsung digunakan sebagai bobot. Implementasi juga menangani fitness negatif dengan menggeser bobot, dan memilih secara seragam apabila semua bobot bernilai nol. Mode seleksi seragam tersedia sebagai opsi eksperimen.

Kode 3.13: Pemilihan induk secara roulette

```
minimum = min(fitnesses)
weights = [fitness - minimum if minimum < 0 else fitness
           for fitness in fitnesses]
if sum(weights) == 0:
    return rng.choice(population)
return rng.choices(population, weights=weights, k=1)[0]
```

3.8.5 Crossover Satu Titik dan Perbaikan Kelayakan

Setelah dua induk dipilih, *single-point crossover* mengambil gen sebelum titik potong dari induk pertama dan gen setelahnya dari induk kedua. Urutan gen mengikuti urutan paket dalam state. Anak mula-mula dibuat dengan semua paket di luar truk, lalu paket-paket yang diwarisi dalam keadaan termuat dicoba satu per satu dari ketinggian terendah. Jika penempatan warisan membuat anak melanggar constraint, program mencoba mencari penempatan valid baru untuk paket itu; jika pencarian perbaikan gagal, paket tetap di luar truk. Dengan demikian, anak yang dihasilkan tetap layak.

Kode 3.14: Mengambil gen dari dua induk pada satu titik potong

```
cut = rng.randrange(1, len(packages)) if len(packages) > 1 else 1
inherited = {
    package.id: (
        parent_a.placements[package.id]
        if index < cut
        else parent_b.placements[package.id]
    )
    for index, package in enumerate(packages)
}
```

Cuplikan tersebut menentukan pewarisan gen. Tahap berikutnya membangun state anak dan memvalidasi setiap penempatan yang diwarisi. Kode di bawah menunjukkan penanganan gen yang tidak dapat langsung dipasang secara valid.

Kode 3.15: Mempertahankan validitas anak hasil crossover

```
candidate = State(packages, parent_a.truck, placements)
if not validate_state(candidate):
    child = candidate
else:
    repaired = random_valid_placement(child, package, rng)
    if repaired is not None:
        placements = dict(child.placements)
        placements[package.id] = repaired
        child = State(packages, parent_a.truck, placements)
```

3.8.6 Mutasi Anak

Setiap anak kemudian mengalami mutasi dengan peluang `mutation_rate`, yang secara default bernilai 0,05. Artinya, peluang tersebut diterapkan pada setiap anak yang dibentuk, bukan secara terpisah pada setiap paket. Jika mutasi terpilih, satu paket dipilih secara acak. Paket yang belum dimuat dicoba untuk dimasukkan; paket yang sudah dimuat dapat dipindahkan, dirotasi, atau dikeluarkan. Kandidat mutasi hanya diterima jika berbeda dari penempatan lama dan seluruh state tetap valid. Fungsi mencoba paling banyak 50 kandidat; jika tidak menemukan mutasi valid, individu tidak berubah.

Kode 3.16: Peluang mutasi pada setiap anak

```
if rng.random() < mutation_rate:
    child = mutate_individual(child, rng)
next_population.append(child)
```

Kode 3.17: Memilih satu paket dan jenis mutasinya

```
package = rng.choice(individual.packages)
old = individual.placements[package.id]
operations = ("insert",) if old is None else ("move", "rotate", "remove")
for _ in range(attempts):
    operation = rng.choice(operations)
    if operation == "remove":
        placement = None
    elif operation == "rotate":
        assert old is not None
        placement = random_valid_placement(
            individual, package, rng,
            orientation=rng.choice(tuple(o for o in ORIENTATIONS if o != old.orientation)),
            coordinate=old.coordinate, attempts=1,
        )
    else:
        placement = random_valid_placement(
            individual, package, rng,
            orientation=old.orientation if old is not None else None,
            attempts=1,
        )
    if placement == old:
        continue
    placements = dict(individual.placements)
    placements[package.id] = placement
    candidate = State(individual.packages, individual.truck, placements)
    if not validate_state(candidate):
        return candidate
return individual
```

3.8.7 Pembentukan Generasi Berikutnya dan Terminasi

Anak-anak dibuat berulang kali sampai jumlahnya sama dengan `population_size`. Populasi baru kemudian menggantikan populasi lama secara generasional; individu terbaik tidak otomatis disalin ke generasi berikutnya, tetapi solusi terbaik global tetap tersimpan dalam `best_state`. Loop mengevaluasi generasi awal bernomor nol hingga generasi ke-G, sehingga menghasilkan $G + 1$ catatan fitness. Setelah generasi terakhir dievaluasi, pencarian berhenti dan hasil akhirnya memuat state dengan fitness terbaik yang pernah dijumpai, riwayat fitness terbaik dan rata-rata per generasi, durasi, seed, serta parameter pencarian.

Kode 3.18: Loop generasi dan pembentukan populasi baru

```
best_history = []
average_history = []
best_state = initial_state
best_score = None

for generation in range(generations + 1):
    fitnesses = _evaluate_population(population, objective)
    best_index = max(range(len(population)), key=fitnesses.__getitem__)
    generation_best = fitnesses[best_index]
    best_history.append(generation_best)
    average_history.append(sum(fitnesses) / len(fitnesses))
    if best_score is None or generation_best > best_score:
        best_state = population[best_index]
        best_score = generation_best
    if generation == generations:
        break

    next_population = []
    while len(next_population) < population_size:
        if selection_mode == "uniform":
            parent_a = rng.choice(population)
            parent_b = rng.choice(population)
        else:
            parent_a = roulette_wheel_select(population, fitnesses, rng)
            parent_b = roulette_wheel_select(population, fitnesses, rng)
        child = (single_point_crossover(parent_a, parent_b, rng)
                 if crossover_enabled else parent_a)
        if rng.random() < mutation_rate:
            child = mutate_individual(child, rng)
        next_population.append(child)
    population = tuple(next_population)
```

Setelah generasi terakhir dievaluasi, algoritma mengemas solusi terbaik dan metrik pencarian ke dalam objek `SearchResult`. Seed, indeks run, dan parameter pencarian turut dicatat agar hasil eksperimen dapat ditelusuri.

Kode 3.19: Mengemas solusi dan metrik hasil pencarian

```
return SearchResult(
    algorithm="genetic_algorithm",
    objective_name=objective_name or ("total_value" if objective_fn is None else "custom"),
    initial_state=initial_state,
    final_state=best_state,
    objective_value=best_score,
    objective_history=tuple(best_history),
    duration_seconds=perf_counter() - started_at,
    metrics=GeneticAlgorithmMetrics(
        generations=generations,
        population_size=population_size,
        best_objective_history=tuple(best_history),
        average_objective_history=tuple(average_history),
    ),
    seed=seed,
    run_index=run_index,
    parameters={
        "population_size": population_size,
        "generations": generations,
        "mutation_rate": mutation_rate,
        "selection_mode": selection_mode,
        "crossover_enabled": crossover_enabled,
    },
)
```

Seed acak diberikan pada generator `Random`, sehingga konfigurasi seed yang sama dapat mereproduksi urutan keputusan acak yang sama. Sebagai catatan implementasi, riwayat fitness terbaik menyimpan nilai terbaik pada masing-masing generasi, sedangkan `final_state` menyimpan individu terbaik sepanjang seluruh generasi. Karena tidak ada elitisme, nilai terbaik per generasi dapat turun meskipun solusi terbaik global yang dikembalikan tetap dipertahankan.

Bab 4

Hasil Eksperimen dan Analisis

4.1 Metodologi Eksperimen

Metodologi eksperimen dirancang untuk membandingkan kualitas solusi dan biaya pencarian secara adil, sekaligus memeriksa ketahanan algoritma pada kasus sulit dan masukan batas. Seluruh algoritma diuji pada instance paket dan truk yang sama, menggunakan fungsi objektif serta validator constraint yang sama. Rincian konfigurasi khusus tiap algoritma dibahas terpisah dan tidak diulang pada metodologi umum ini.

4.1.1 Kumpulan Instance dan Kasus Uji

Kumpulan uji mencakup instance utama dari data tugas serta instance terkontrol yang dibentuk untuk menguji karakteristik tertentu. Ukuran dan tingkat kesulitan instance dibuat bervariasi agar hasil tidak hanya menggambarkan satu konfigurasi pemuatan. Setidaknya, pengujian mencakup kategori berikut:

(1) **Kasus dasar.** Instance dengan paket yang bervariasi dalam dimensi, nilai, dan berat, serta beberapa kombinasi pemuatan yang layak. Kasus ini digunakan untuk membandingkan kinerja umum.

(2) **Kasus kendala ketat.** Ruang truk diperkecil atau kapasitas berat dikurangi agar lebih banyak kandidat penempatan ditolak. Instance ini menguji dampak keterbatasan ruang dan kapasitas.

(3) **Kasus tanpa pemuatan nonkosong yang layak.** Semua paket dibuat tidak dapat dimuat, misalnya setiap paket lebih besar daripada ruang truk pada semua orientasi atau beratnya melebihi kapasitas. Karena representasi tetap mengizinkan semua paket berada di luar truk, state kosong merupakan state valid dengan nilai objektif nol; kasus ini menguji apakah algoritma dapat mengembalikan state valid tanpa paket termuat.

(4) **Kasus batas dan struktur khusus.** Pengujian meliputi satu paket, paket yang hanya muat setelah rotasi, paket yang tepat menyentuh dinding truk, berat total yang tepat sama dengan kapasitas, paket rapuh yang memengaruhi dukungan, serta paket bernilai atau berbobot nol. Kasus dengan fitness sama juga digunakan untuk memeriksa perilaku ketika seleksi tidak memiliki preferensi nilai.

(5) **Masukan tidak valid.** Data dengan dimensi atau kapasitas yang melanggar format, ID paket duplikat, atau daftar paket kosong diuji pada batas validasi data. Kasus ini dipisahkan dari benchmark pencarian karena bukan instance masalah yang sah menurut loader.

Instance sintetis untuk uji batas dibuat dengan mengubah satu karakteristik pada satu waktu dari instance dasar, sementara atribut lain dipertahankan. Dengan demikian, perbedaan hasil dapat dikaitkan dengan kendala yang sedang diuji. Kasus yang dirancang untuk memastikan tidak ada paket yang dapat dimuat dibedakan secara eksplisit dari kasus sulit yang masih memiliki solusi layak.

4.1.2 Protokol Perbandingan

Untuk setiap pasangan instance dan seed, state awal yang sama digunakan oleh semua algoritma. Daftar seed ditetapkan sebelum eksperimen; setiap kombinasi dijalankan sedikitnya tiga kali dengan seed berbeda sesuai kebutuhan evaluasi tugas. Jika anggaran komputasi memungkinkan, jumlah pengulangan ditambah agar estimasi variasi hasil lebih stabil. Penggunaan seed dan state awal berpasangan mengurangi pengaruh variasi inisialisasi saat membandingkan metode.

Selain jenis instance, faktor lain seperti data paket, truk, fungsi objektif, dan implementasi validasi dijaga tetap sama. Batas penghentian dan konfigurasi spesifik mengikuti definisi masing-masing algoritma; karena satu iterasi pada algoritma berbeda belum tentu setara, jumlah iterasi tidak digunakan sendirian sebagai ukuran biaya lintas algoritma. Durasi pencarian dan jumlah evaluasi fungsi objektif dicatat sebagai ukuran biaya yang lebih langsung. Pengukuran waktu dilakukan pada lingkungan komputasi yang sama, dengan waktu persiapan data dan pembuatan visualisasi dipisahkan dari waktu pencarian.

Setiap hasil akhir diperiksa kembali menggunakan validator. Run dinyatakan berhasil jika algoritma mengembalikan state valid; pada instance tanpa pemuatan nonkosong yang layak, state akhir harus valid dan tidak memuat paket. State awal dan akhir, seed, nilai objektif, jumlah paket termuat, riwayat pencarian, durasi, serta konfigurasi run disimpan agar hasil dapat direproduksi dan ditelusuri.

4.1.3 Metrik dan Analisis

Kualitas hasil dinilai dari nilai objektif akhir dan jumlah paket yang termuat. Ketahanan terhadap constraint dinilai dari proporsi run yang menghasilkan state akhir valid. Konsistensi diukur dengan merangkum nilai objektif dan durasi antarrun menggunakan rata-rata serta simpangan baku; hasil tiap run tetap disajikan agar variasi tidak tersembunyi oleh agregat. Riwayat nilai objektif juga ditampilkan untuk memperlihatkan perkembangan pencarian. Untuk perbandingan lintas algoritma, kurva dapat disejajarkan terhadap jumlah evaluasi objektif atau waktu berjalan, selain menampilkan sumbu iterasi atau generasi yang diwajibkan untuk masing-masing algoritma.
Pada kasus tanpa pemuatan nonkosong yang layak, nilai objektif nol saja tidak cukup untuk menyimpulkan perilaku algoritma karena paket bernilai nol dapat saja termuat. Oleh sebab itu, jumlah paket termuat dan validitas state diperiksa bersamaan dengan nilai objektif. Hasil kasus ini dilaporkan sebagai uji ketahanan, terpisah dari perbandingan kualitas solusi pada instance yang dapat dimuat.

4.1.4 Studi Ablasi

Studi ablasi digunakan untuk mengetahui kontribusi pilihan desain terhadap hasil pencarian. Untuk setiap studi, satu konfigurasi acuan dibandingkan dengan beberapa varian konfigurasi. Pada tiap perbandingan, hanya faktor yang sedang diuji yang diubah; instance, state awal, seed, fungsi objektif, constraint, dan anggaran pencarian lainnya dijaga tetap sama. Jika dua faktor diduga saling memengaruhi, interaksinya diuji pada eksperimen terpisah, bukan diubah sekaligus tanpa kontrol.
Setiap varian dijalankan pada seed yang sama dengan konfigurasi acuannya, sehingga hasil dapat dibandingkan secara berpasangan. Analisis melaporkan perubahan nilai objektif akhir dan durasi terhadap acuan, disertai rata-rata dan simpangan baku serta riwayat pencarian. Dengan protokol ini, studi ablasi menilai apakah perubahan konfigurasi memberi manfaat yang konsisten, bukan hanya hasil yang lebih baik pada satu seed.

4.2 Hasil Eksperimen dan Analisis tiap Algoritma

4.2.1 Hill Climbing

4.3 Hasil Eksperimen dan Analisis tiap Algoritma

4.3.1 Hill Climbing

4.3.2 Simulated Annealing

4.3.3 Genetic Algorithm

4.4 Analisis Keseluruhan

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

5.2 Saran

Bab 6

Lampiran

6.1 Pembagian Tugas

6.2 Repositori GitHub

Bibliografi

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[2] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[3] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[4] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 2nd ed. Cambridge University Press, 2017. [Online]. Available: https://artint.info/2e/html2e/ArtInt2e.Ch4.S7.SS1.html